

Indeksy, optymalizator

Lab 2

Imię i nazwisko:

- Paweł Moćko
 - Katarzyna Wesołowska
-

Celem ćwiczenia jest zapoznanie się z planami wykonania zapytań (execution plans), oraz z budową i możliwością wykorzystaniem indeksów (kontynuacja poprzedniego ćwiczenia)

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

Wyniki:

-- ...

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie

- MS SQL Server
- SSMS - SQL Server Management Studio
 - ewentualnie inne narzędzie umożliwiające komunikację z MS SQL Server i analizę planów zapytań

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Przygotowanie

Uruchom Microsoft SQL Managment Studio.

Stwórz swoją bazę danych o nazwie lab2.

```
create database lab2
go

use lab2
go
```

Warto przełączyć bazę w tryb simple

```
alter database lab2  
set recovery simple;
```

Zadanie 1 - indeksy

Wykonaj poniższy skrypt, aby przygotować dane:

```
select * into product_history
from northwind3.dbo.product_history

select * into categories
from northwind3.dbo.categories

create clustered index categ_clust_idx
on categories(categoryid)
```

sprawdź liczbę wierszy w tabeli

```
select count(*) from product_history
```

Sprawdź jakie indeksy istnieją dla tej tabeli

```
exec sp_helpindex 'dbo.product_history'
```

```
Select
    i.name as index_name,
    i.type_desc,
    i.is_unique,
    c.name as column_name,
    ic.key_ordinal,
    ic.is_included_column
from sys.indexes i
join sys.index_columns ic
    on i.object_id = ic.object_id
    and i.index_id = ic.index_id
join sys.columns c
    on ic.object_id = c.object_id
    and ic.column_id = c.column_id
where i.object_id = object_id('dbo.product_history')
order by i.name, ic.key_ordinal;
```

włącz statystyki IO i TIME

```
SET STATISTICS IO ON  
  
SET STATISTICS TIME ON;
```

podczas analiz sprawdzaj jak zachowują się zapytania, zwróć uwagę na

- plan
- koszt
- czas (ewentualnie, jeśli coś da się zaobserwować)
- liczbę odczytywanych stron !!!!

porównaj zapytania

a)

```
select count(*) from product_history  
where id = 1000000  
  
select count(*) from product_history  
where id between 999000 and 10000000
```

Komentarz

- pierwsze zapytanie zwróciło wynik 1.
- mimo że szukany był tylko jeden rekord, SQL Server przeszukał całą tabelę.
- widać to po planie wykonania (Skan tabeli) i po bardzo dużej liczbie odczytanych stron (logical reads = 25265).
- koszt skanu tabeli wyniósł około 19,35, a koszt całego zapytania około 19,65.
- czas wykonania wyniósł około 216 ms.
- drugie zapytanie zwróciło 1311001.
- SQL Server znowu przeszukał całą tabelę.
- liczba odczytanych stron była taka sama jak wcześniej.
- koszt skanu tabeli również wyniósł około 19,35, a koszt całego zapytania około 19,89.
- czas wykonania był większy i wyniósł około 436 ms.

Porównanie:

- plan oraz liczba odczytanych stron obu zapytań były bardzo podobne, ponieważ w obu przypadkach SQL Server wykonał skan tabeli.
- koszt zapytań był bardzo zbliżony, ponieważ sposób wykonania był prawie identyczny.
- czas wykonania był większy dla zapytania zakresowego, bo zwróciło ono dużo więcej rekordów.

b)

```
select * from product_history
where id = 1000000

select * from product_history
where id between 999000 and 10000000
```

Komentarz

- zapytanie zwróciło 1 pełny wiersz.
- mimo że szukany był tylko jeden rekord, SQL Server znowu wykonał skan tabeli.
- liczba odczytanych stron była taka sama jak wcześniej: logical reads = 25265.
- koszt skanu tabeli wyniósł około 19,35, a koszt całego zapytania około 19,66.
- czas wykonania był większy niż przy count(*) i wyniósł około 354 ms, ponieważ tutaj trzeba było odczytać cały rekord, a nie tylko policzyć wynik.
- drugie zapytanie zwróciło 1311001 wierszy.
- SQL Server znowu przeszukał całą tabelę.
- liczba odczytanych stron była taka sama jak wcześniej.
- koszt całego zapytania wyniósł około 21,26, więc był wyższy niż w pierwszej części.
- czas wykonania był bardzo duży i wyniósł około 30442 ms, ponieważ zapytanie zwróciło dużo wierszy.

Porównanie:

- w obu zapytaniach SQL Server wykonał skan tabeli oraz liczba odczytanych stron była taka sama, bo serwer musiał przeczytać całą tabelę.
- zapytanie dla pojedynczej wartości miało mniejszy koszt i dużo krótszy czas wykonania.
- zapytanie dla zakresu było znacznie cięższe, bo zwróciło bardzo dużo pełnych wierszy.

c)

sprawdź jak zachowują się zapytania z pkt a) i b) jeśli dla kolumny `id` stworzysz indeks

- klastrowy
- nieklastrowy

```
create clustered index product_history_clust_idx
on product_history(id)

drop index product_history_clust_idx on product_history

create index product_history_idx
on product_history(id)

drop index product_history_idx on product_history
```

po zakończeniu pozostaw indeks klastrowy

Komentarz

Indeks klastrowy:

- podpunkt a, część 1:
 - po założeniu indeksu klastrowego zapytanie korzysta z indeksu zamiast skanować całą tabelę
 - logical reads spadły z 25265 do 3, a czas do około 3 ms
 - koszt spadł z około 19,65 do około 0,003
- podpunkt a, część 2:
 - zapytanie również korzysta z indeksu klastrowego
 - logical reads spadły z 25265 do 14794, a czas do około 180 ms
 - koszt spadł z około 19,89 do około 11,26
- podpunkt b, część 1:
 - po założeniu indeksu klastrowego zapytanie bardzo szybko odnalazło jeden rekord
 - logical reads spadły z 25265 do 3, a czas do około 4 ms
 - koszt spadł z około 19,66 do około 0,003
- podpunkt b, część 2:
 - zapytanie nadal zwraca bardzo dużo danych, więc pozostaje kosztowne
 - mimo to logical reads spadły z 25265 do 14692, a czas z około 30442 ms do około 22662 ms
 - koszt spadł z około 21,26 do około 12,31

Podsumowanie:

- po utworzeniu indeksu klastrowego zapytania przestały wykonywać skan całej tabeli i zaczęły korzystać z indeksu na kolumnie id.
- największa poprawa była widoczna dla zapytań wyszukujących jeden konkretny rekord dla podanego id.
- dla zapytań z zadaniem zakresem poprawa też była widoczna, ale mniejsza, ponieważ nadal trzeba było odczytać dużo danych.
- czas oraz liczba odczytanych stron jest znacznie mniejsza.
- koszt zapytań dla podanego id spadł bardzo mocno, natomiast dla zapytań z podanym zakresem również spadł, ale nie tak wyraźnie jak dla pojedynczej wartości.
- indeks klastrowy najlepiej pomaga wtedy, gdy wyszukujemy po id mały fragment danych albo pojedynczy rekord.

Indeks nieklastrowy:

- podpunkt a, część 1:
 - logical reads = 3, czas około 8 ms, koszt około 0,003
- podpunkt a, część 2:
 - logical reads = 2934, czas około 303 ms, koszt około 4,40
- podpunkt b, część 1:
 - indeks nieklastrowy został użyty, a potem serwer dobrał pełny wiersz z tabeli
 - logical reads = 4, czas około 8 ms, koszt około 0,0066
- podpunkt b, część 2:
 - SQL Server nie skorzystał już sensownie z indeksu i wrócił do skanowania całej

tabeli

- logical reads = 25837, czas około 23439 ms, koszt około 21,68

Podsumowanie:

- dla zapytań wyszukujących jeden konkretny rekord dla podanego id, indeks nieklastrowy bardzo pomógł.
- w planie wykonania dla zapytań punktowych pojawiło się wyszukiwanie indeksu nieklastrowego
- najgorzej wypadło zapytanie `select *` dla dużego zakresu. W tym przypadku SQL Server wrócił do skanowania całej tabeli.
- indeks nieklastrowy jest dobry dla zapytań punktowych i mniejszych zakresów, ale gorzej sprawdza się przy dużym zakresie i `select *`.

d)

indeks dla kolumny `date`

```
create index product_history_date_idx
on product_history(date)

drop index product_history_date_idx on product_history
```

porównaj polecenia

```
select id, productid, productname, date
from product_history
where date >= '2001-01-01' and date <= '2001-01-31'

select id, productid, productname, date
from product_history
where year(date) = 2001 and month(date) = 1

select id, productid, productname, date
from product_history
where date >= '2001-01-01' and date <= '2001-12-31'

select id, productid, productname, date
from product_history
where year(date) = 2001
```

podczas analiz sprawdzaj jak zachowują się zapytania, zwróć uwagę na

- plan
- indeksy i sposób ich użycia
- koszt
- czas (ewentualnie, jeśli coś da się zaobserwować)
- liczbę odczytywanych stron !!!!

spróbuj skomentować wyniki tych analiz, dlaczego tak się dzieje

Komentarz:

- Dla pierwszego zapytania: SQL Server skorzystał z indeksu na kolumnie date. Dzięki temu liczba odczytanych stron była dużo mniejsza (logical reads = 7327), a czas wykonania wyniósł około 40 ms.
- Dla drugiego zapytania: SQL Server nie użył już tego indeksu w dobry sposób i wykonał skanowanie. Liczba odczytanych stron wzrosła do 26057, a czas wykonania do około 318 ms.
- Dla trzeciego zapytania: SQL Server również wykonał skanowanie. W tym przypadku zakres danych był już na tyle duży, że użycie indeksu nie dawało wyraźnej korzyści. Liczba odczytanych stron wyniosła 26057, a czas około 151 ms.
- Dla czwartego zapytania: plan był bardzo podobny jak wyżej: znowu pojawiło się skanowanie, a liczba odczytanych stron była taka sama (26057). Czas wykonania wyniósł około 181 ms.

Porównanie:

- najlepiej zadziałało zapytanie z dokładnym zakresem dat dla jednego miesiąca, bo SQL Server mógł użyć indeksu na kolumnie date.
- zapytania z funkcjami year(date) i month(date) w warunku wypadły gorzej, ponieważ taki zapis utrudnia wykorzystanie indeksu.
- dla dużego zakresu, obejmującego cały rok, SQL Server i tak wybrał skanowanie, bo trzeba było odczytać dużą część tabeli.
- wniosek jest taki, że indeks na date najlepiej pomaga wtedy, gdy warunek odwołuje się bezpośrednio do kolumny date, a nie gdy na tej kolumnie najpierw wykonuje funkcje year() i month().

e)

powtórz eksperymenty z pkt d) , ale tym razem użyj indeksu zawierającego dodatkowe kolumny

```
create index product_history_date_incl_idx
on product_history(date) include(productid, productname)

drop index product_history_date_incl_idx on product_history
```

co się zmieniło?

Komentarz

- po utworzeniu indeksu z INCLUDE zapytania działały lepiej niż w punkcie d).
- największa poprawa była dla zapytań z warunkiem zakresowym po date.
- liczba odczytanych stron bardzo spadła, np. dla jednego miesiąca z 7327 do 16, a dla całego roku z 26057 do 141.
- zapytania z YEAR(date) i MONTH(date) nadal działały gorzej niż zapytania zakresowe, ale i tak były lepsze niż wcześniej.
- indeks z INCLUDE okazał się lepszy, ponieważ zawiera dodatkowe kolumny potrzebne w SELECT.

Co się zmieniło?

- SQL Server mógł odczytać więcej danych bez dodatkowego wracania do tabeli.
- zapytania zakresowe działały wyraźnie szybciej i czytały dużo mniej stron.
- zapytania z funkcjami nadal były mniej efektywne.

f)

indeks dla kolumny `categoryid`

```
create index product_history_cat_idx
on product_history(categoryid)

drop index product_history_cat_idx on product_history
```

przeanalizuj polecenia

```
select id, productid, productname, date
from product_history p
where categoryid = 8

select id, productid, productname, date, categoryname
from product_history p join categories c on p.categoryid = c.categoryid
where p.categoryid = 8
```

Komentarz:

- indeks na `categoryid` nie dał dużej poprawy, ponieważ warunek `categoryid = 8` zwraca bardzo dużo rekordów.
- w obu zapytaniach największy koszt dotyczył tabeli `product_history`.
- tabela `categories` była mała, więc problemem nie było złączenie, tylko liczba danych w `product_history`.
- sam indeks na `categoryid` okazał się niewystarczający.

dodatkowo

możesz sprawdzić strukturę indeksu

np.

```
exec sp_helpindex 'dbo.product_history';

select
    i.name as index_name,
    ips.index_depth,
    ips.index_level,
    ips.page_count
from sys.indexes i
cross apply sys.dm_db_index_physical_stats(
    db_id(),
    i.object_id,
    i.index_id,
    null, 'detailed'
) ips
where i.object_id = object_id('dbo.product_history')
and i.name = 'product_history_date_idx';
```

jeśli chcesz zaobserwować odczyty logiczne/fizyczne możesz zwolnić pulę buforów przed wykonaniem polecenia

```
CHECKPOINT;
DBCC DROPCLEANBUFFERS;
```

i teraz porównaj liczby czytanych stron np. wykonując dwukrotnie polecenie

```
select * from product_history
```

Komentarz:

- oba wykonania zapytania miały taki sam plan (zawierające skanowanie indeksu klastrowego) i taką samą liczbę logical reads (25883)
- różnica pojawiła się w physical reads i read-ahead reads.
- przy pierwszym wykonaniu po wyczyszczeniu bufora wystąpiły odczyty fizyczne (physical reads = 1, read-ahead reads = 25893), a przy drugim już ich nie było.
- oznacza to, że po pierwszym uruchomieniu dane znalazły się w pamięci, dlatego drugie wykonanie korzystało już z bufora.

Tabela podsumowująca:

Punkt	Wariant	Zapytanie	Najważniejsze elementy planu	Koszt	elapsed time	logical reads
a1	bez indeksu na <code>id</code>	<code>count(*) where id = 1000000</code>	Skan tabeli	19.65	216 ms	25265
a2	bez indeksu na <code>id</code>	<code>count(*) where id between 999000 and 10000000</code>	Skan tabeli	19.89	436 ms	25265
b1	bez indeksu na <code>id</code>	<code>select * where id = 1000000</code>	Skan tabeli	19.66	354 ms	25265
b2	bez indeksu na <code>id</code>	<code>select * where id between 999000 and 10000000</code>	Skan tabeli	21.26	30442 ms	25265
c / a1	indeks klastrowy na <code>id</code>	<code>count(*) where id = 1000000</code>	Clustered Index Seek + agregacja	0.0032842	3 ms	3
c / a2	indeks klastrowy na <code>id</code>	<code>count(*) where id between 999000 and 10000000</code>	Clustered Index Seek + agregacja	11.2587	180 ms	14794
c / b1	indeks klastrowy na <code>id</code>	<code>select * where id = 1000000</code>	Clustered Index Seek	0.0032831	4 ms	3

Punkt	Wariant	Zapytanie	Najważniejsze elementy planu	Koszt	elapsed time	logical reads
c / b2	indeks klastrowy na id	<code>select * where id between 999000 and 10000000</code>	Clustered Index Seek	12.3069	22662 ms	14692
c / a1	indeks nieklastrowy na id	<code>count(*) where id = 1000000</code>	Wyszukiwanie indeksu nieklastrowego + agregacja	0.0032842	8 ms	3
c / a2	indeks nieklastrowy na id	<code>count(*) where id between 999000 and 10000000</code>	Wyszukiwanie indeksu nieklastrowego + agregacja	4.39569	303 ms	2934
c / b1	indeks nieklastrowy na id	<code>select * where id = 1000000</code>	Wyszukiwanie indeksu nieklastrowego + RID Lookup	0.0065704	8 ms	4
c / b2	indeks nieklastrowy na id	<code>select * where id between 999000 and 10000000</code>	Skan tabeli	21.6821	23439 ms	25837
d	indeks na date	zakres styczeń 2001	Wyszukiwanie indeksu + Key Lookup	7.51795	40 ms	7327
d	indeks na date	<code>YEAR(date)=2001 AND MONTH(date)=1</code>	Skanowanie indeksu klastrowego	20.3818	318 ms	26057
d	indeks na date	zakres cały 2001	Skanowanie indeksu klastrowego	20.3513	151 ms	26057
d	indeks na date	<code>YEAR(date)=2001</code>	Skanowanie indeksu klastrowego	20.1782	181 ms	26057
e	indeks na date + <code>INCLUDE(productid, productname)</code>	zakres styczeń 2001	Wyszukiwanie indeksu nieklastrowego	0.0140558	16 ms	16
e	indeks na date + <code>INCLUDE(productid, productname)</code>	<code>YEAR(date)=2001 AND MONTH(date)=1</code>	Skanowanie indeksu nieklastrowego	9.57955	136 ms	11409
e	indeks na date + <code>INCLUDE(productid, productname)</code>	zakres cały 2001	Wyszukiwanie indeksu nieklastrowego	0.134938	96 ms	141
e	indeks na date + <code>INCLUDE(productid, productname)</code>	<code>YEAR(date)=2001</code>	Skanowanie indeksu nieklastrowego	9.37599	190 ms	11409

Punkt	Wariant	Zapytanie	Najważniejsze elementy planu	Koszt	elapsed time	logical reads
						<code>productname)</code>
f	indeks na <code>categoryid</code>	<code>where categoryid = 8</code>	Skanowanie indeksu klastrowego na <code>product_history</code>	20.5734	3808 ms	26009
f	indeks na <code>categoryid</code>	<code>join categories ... where p.categoryid = 8</code>	Skanowanie indeksu klastrowego na <code>product_history</code> + zagnieżdżone pętle + odczyt z <code>categories</code>	23.8679	3708 ms	25883 + 2

Wnioski końcowe:

Brak indeksu powodował, że SQL Server w każdym przypadku przeszukiwał całą tabelę, nawet wtedy, gdy zapytanie dotyczyło tylko jednego rekordu. Po utworzeniu indeksu klastrowego na id zapytania działały dużo lepiej, szczególnie dla wyszukiwania pojedynczej wartości. Dla większych zakresów też było lepiej, ale poprawa była mniejsza, bo nadal trzeba było odczytać dużo danych. Indeks nieklastrowy na id również dobrze pomagał przy prostych zapytaniach i przy `count(*)`, ale przy `select *` dla dużego zakresu okazał się słabszy. W takim przypadku SQL Server i tak wracał do czytania bardzo dużej części tabeli. Dla kolumny date najlepiej działały zapytania zapisane jako zakres dat, na przykład od początku do końca miesiąca. Zapytania z `YEAR(date)` i `MONTH(date)` działały gorzej, bo SQL Server nie potrafił wtedy tak dobrze wykorzystać indeksu. Po dodaniu indeksu z `INCLUDE` odczytów było jeszcze mniej i zapytania działały szybciej. W przypadku `categoryid` sam indeks nie dał dużej poprawy, ponieważ warunek zwracał bardzo dużo wierszy. To pokazuje, że indeks najbardziej pomaga wtedy, gdy dobrze zawęży dane, a nie wtedy, gdy i tak trzeba odczytać bardzo dużą część tabeli.

Zadanie 2

Celem zadania jest poznanie indeksów typu column store

Utwórz tabelę testową:

```
create table saleshistory(
  id int identity(1,1) not null primary key,
  salesorderid int not null,
  salesorderdetailid int not null,
  carriertrackingnumber nvarchar(25) null,
  orderqty smallint not null,
  productid int not null,
  specialofferid int not null,
  unitprice money not null,
  unitpricediscount money not null,
  linetotal numeric(38, 6) not null,
  rowguid uniqueidentifier not null,
  modifieddate datetime not null
)
```

Sprawdź jakie indeksy istnieją dla tej tabeli

```
exec sp_helpindex 'dbo.saleshistory'
```

```
Select
    i.name as index_name,
    i.type_desc,
    i.is_unique,
    c.name as column_name,
    ic.key_ordinal,
    ic.is_included_column
from sys.indexes i
join sys.index_columns ic
    on i.object_id = ic.object_id
    and i.index_id = ic.index_id
join sys.columns c
    on ic.object_id = c.object_id
    and ic.column_id = c.column_id
where i.object_id = object_id('dbo.saleshistory')
order by i.name, ic.key_ordinal;
```

Wynik:

index_name	type_desc	is_unique	column_name	key_ordinal	is_included_column
PK_saleshis__3213E83F84356F2C	CLUSTERED	true	id	1	false

Istnieje jeden indeks, dla
kolumny tworzącej klucz główny
tabeli.

Wypełnij tablicę danymi:

```
-- W SSMS

insert into saleshistory
select sh.*
from adventureworks2017.sales.salesorderdetail sh
go 100
```

(UWAGA GO 100 oznacza 100 krotne wykonanie polecenia. Jeżeli podejrzewasz, że twój serwer może to zbyt przeciążyć, zacznij od GO 10, GO 20, GO 50

albo

```
declare @i int = 1;

while @i <= 100
begin
```

```
insert into saleshistory
select *
from adventureworks2017.sales.salesorderdetail;

set @i += 1;
end;
```

sprawdź liczbę wierszy w tabeli

```
select count(*) from saleshistory
```

Zgodnie z podpowiedzią niżej, wygenerowano większy wolumen danych poprzez wykonanie operacji dodania danych dwukrotnie. Wynik powyższego zapytania:

24263400

włącz statystyki IO i TIME

```
SET STATISTICS IO ON

SET STATISTICS TIME ON;
```

Sprawdź jak zachowa się zapytanie

- sprawdź plan
- koszt
- czas
- liczbę odczytywanych stron

```
select productid, sum(unitprice), avg(unitprice), sum(orderqty), avg(orderqty)
from saleshistory
group by productid
order by productid
```

Załącz indeks typu column store:

```
create nonclustered columnstore index saleshistory_columnstore
on saleshistory(unitprice, orderqty, productid)
```

Sprawdź różnicę pomiędzy przetwarzaniem w zależności od indeksów. Porównaj plany i opisz różnicę.

Porównanie

Porównujemy plan wykonania, koszt, czas oraz liczbę odczytywanych stron przed i po stworzeniu indeksu używając powyższego zapytania.

W celu porównania czasu wykonania, dokonaliśmy czterech wywołań zapytania i podajemy uśredniony wynik z trzech ostatnich.

indeks	czas [ms]	koszt	odczyty logiczne	odczyty fizyczne
domyślny	865	236.583	317717	2
nonclustered columnstore	66	2.447	24711	44

Wykorzystanie indeksu zredukowało czas wykonania ~131 razy, a liczbę odczytów logicznych ~12.85 razy.

Liczba odczytów logicznych

Liczba stron w pierwszym przypadku sprawiała wrażenie wysokiej. Dla uwierzytelnienia wyników sprawdziliśmy stosunek **liczba wierszy / liczba stron**, wynik to ok. 76. W MS SQL jedna strona mieści 8 KB danych, zatem jeden wiersz powinien wówczas zajmować ok. 0.105 KB pamięci. Następnie wykonaliśmy polecenie

```
EXEC sp_spaceused 'saleshistory';
```

które zwróciło nam liczbę przestrzeni dyskowej potrzebnej na przechowanie tabeli. Stosunek **pamięć / liczba wierszy** wynosi ok. 0.102 KB. Ta dygresja dała nam lepsze spojrzenie na to jak dużo danych (wierszy) znajduje się na jednej stronie.

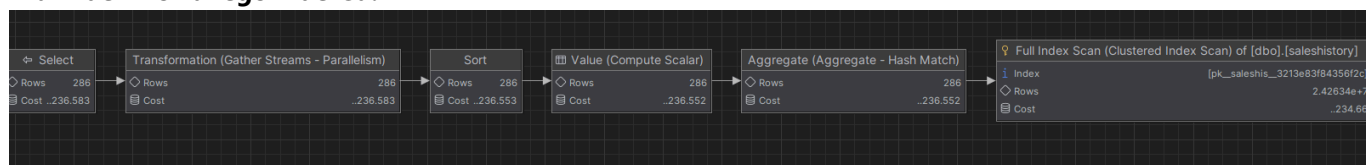
Liczba odczytów dla indeksu columnstore

W pełnym raporcie dla zapytania przy użyciu **SHOW STATISTICS IO ON** Informacje o odczytach logicznych oraz fizycznych znajdują się w sekcji dotyczącej LOB (large objects), zamiast "logical / physical reads"².

```
[2026-05-10 20:59:10] [S0000][3615] Table 'saleshistory'. Scan count 24, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 24711, lob physical reads 44, lob page server reads 0, lob read-ahead reads 92142, lob page server read-ahead reads 0.
```

plany wykonania, czym są indeksy columnstore?

Brak zdefiniowanego indeksu:



Wykorzystany jest domyślny indeks (clustered, po kolumnie **id**). Wydaje się to sprowadzać do pełnego skanu wg wierszy. **Nonclustered columnstore** Po utworzeniu indeksu widzimy znaczną redukcję kosztu dla zapytania. Wykorzystanie columnstore pozwala zawęzić operacje odczytu do kolumn, które są wymagane dla zapytania. Redukuje to znacznie czas na operacje I/O. Przechowywanie danych z podziałem na kolumny pozwala na dodatkową kompresję danych na dysku (i dodatkową redukcję operacji I/O)¹.

UWAGA: ciekawsze efekty możesz zaobserwować dla jeszcze większych tabel (jeśli twój komp na to pozwala możesz zwiększyć wolumen generowanych danych)

Zadanie 3 – własne eksperymenty

Należy zaprojektować/zaimplementować tabelę w bazie danych, lub wybrać dowolny schemat/bazę/tabelę (poza używanymi na zajęciach), a następnie wypełnić ją danymi w taki sposób, przetestować/przeanalizować działanie indeksów różnego typu. Warto wygenerować sobie tabele o większym rozmiarze.

Możesz też powtórzyć np. eksperymenty wykonywane w zadaniu 1, ale tym razem dla innego serwera,

Wedle uznania i zainteresowań, ważne żeby poeksplorować tematykę i spróbować

Do analizy, proszę uwzględnić następujące rodzaje indeksów:

- Klastrowane (np. dla atrybutu nie będącego kluczem głównym)
- Nieklastrowane
- Indeksy wykorzystujące kilka atrybutów, indeksy include
- Filtered Index (Indeks warunkowy)
- Kolumnowe

Analiza

Proszę przygotować zestaw zapytań do danych, które:

- wykorzystują poszczególne indeksy
- które przy wymuszeniu indeksu działają gorzej, niż bez niego (lub pomimo założonego indeksu, tabela jest w pełni skanowana) Odpowiedź powinna zawierać:
- Schemat tabeli
- Opis danych (ich rozmiar, zawartość, statystyki)
- Opis indeksu
- Przygotowane zapytania, wraz z wynikami z planów (zrzuty ekranów)
- Inf o kosztach, czytanych stornach
- Komentarze do zapytań, ich wyników
- ew. sprawdzenie, co proponuje Database Engine Tuning Advisor (porównanie czy udało się Państwu znaleźć odpowiednie indeksy do zapytania)

Wyniki:

Tworzenie tabeli w celu analizy

W celach eksperymentów wykorzystaliśmy bazę danych WideWorldImporters³, którą zaimportowaliśmy w następujący sposób:

```
RESTORE DATABASE WideWorldImporters
FROM DISK = '/mssql/data/WideWorldImporters-Full.bak'
WITH
    MOVE 'WWI_Primary'
        TO '/var/opt/mssql/data/WideWorldImporters.mdf',

    MOVE 'WWI_UserData'
        TO '/var/opt/mssql/data/WideWorldImporters_UserData.ndf',
```



```
MOVE 'WWI_Log'
    TO '/var/opt/mssql/data/WideWorldImporters.ldf',

MOVE 'WWI_InMemory_Data_1'
    TO '/var/opt/mssql/data/WideWorldImporters_InMemory_Data_1',

RECOVERY,
REPLACE;-- ...
```

Cała baza danych zawiera wiele kolumn. Na potrzeby analizy ograniczyliśmy znacznie zakres danych poniższymi zapytaniami:

```
use lab2;

drop table if exists dbo.lab2_sales;

select
    il.InvoiceLineID,
    il.InvoiceID,
    i.CustomerID,
    i.InvoiceDate,
    il.StockItemID,
    il.PackageTypeID,
    il.Quantity,
    il.UnitPrice,
    il.LineProfit,
    il.ExtendedPrice
into dbo.lab2_sales
from WideWorldImporters.Sales.InvoiceLines il
join WideWorldImporters.Sales.Invoices i
    on il.InvoiceID = i.InvoiceID;
```

Schemat tabeli:

column_name	data_type	is_nullable
InvoiceLineID	int	NO
InvoiceID	int	NO
CustomerID	int	NO
InvoiceDate	date	NO
StockItemID	int	NO
PackageTypeID	int	NO
Quantity	int	NO
UnitPrice	decimal(18,2)	YES
LineProfit	decimal(18,2)	NO
ExtendedPrice	decimal(18,2)	NO

Sprawdziliśmy też statystyki dla niektórych kolumn:

```
select
    count(*) as row_count,
    count(distinct InvoiceID) as distinct_invoices,
    count(distinct CustomerID) as distinct_customers,
    count(distinct StockItemID) as distinct_stock_items,
    count(distinct PackageTypeID) as distinct_package_types,
    min(InvoiceDate) as min_invoice_date,
    max(InvoiceDate) as max_invoice_date,
    min(ExtendedPrice) as min_extended_price,
    avg(ExtendedPrice) as avg_extended_price,
    max(ExtendedPrice) as max_extended_price
from dbo.lab2_sales;
```

row_count	228265
distinct_invoices	70510
distinct_customers	663
distinct_stock_items	227
distinct_package_types	4
min_invoice_date	2013-01-01
max_invoice_date	2016-05-31
min_extended_price	2.88
avg_extended_price	867.603178
max_extended_price	21838.50

Porównanie indeksów

Zaczęliśmy od skonstruowania zapytania, które wykorzystamy do porównania różnych rodzajów indeksów.

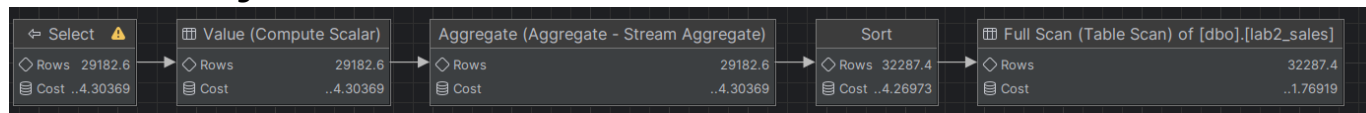
```
SELECT CustomerID,
    StockItemID,
    sum(Quantity) AS TotalQuantity,
    avg(UnitPrice) AS AvgUnitPrice,
    sum(LineProfit) AS TotalProfit,
    sum(ExtendedPrice) AS TotalRevenue
FROM dbo.lab2_sales
WHERE InvoiceDate >= '2015-01-01'
    AND InvoiceDate < '2016-01-01'
    AND ExtendedPrice >= 1000
GROUP BY CustomerID, StockItemID
ORDER BY CustomerID, StockItemID;
```

Zapytanie zostało tak skonstruowane, aby:

- zawierało filtry według wielu kolumn
- odczytywało wiele kolumn
- zawarte w nim było grupowanie danych Następnie skonstruowaliśmy serię indeksów:

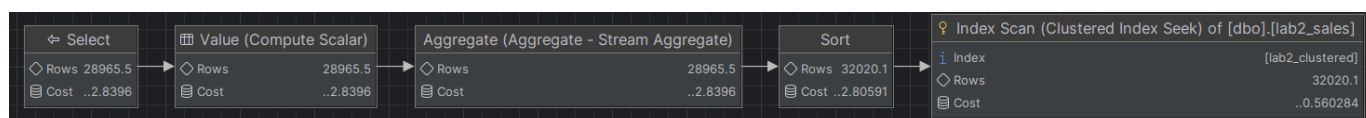
plany wykonań

Brak zdefiniowanego indeksu

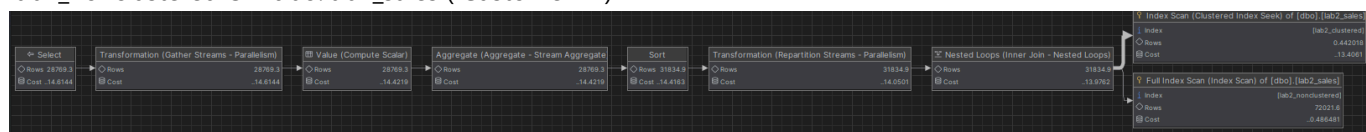


Powyżej mamy full scan - możemy traktować jako baseline do porównań. Dla wyższych kosztów, możemy stwierdzić że indeks spowalnia działanie. **clustered**

```
CREATE CLUSTERED INDEX lab2_clustered
ON dbo.lab2_sales ( InvoiceDate );
```

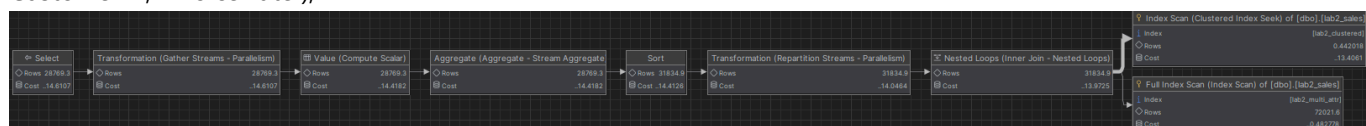


Tutaj wykorzystaliśmy indeks typu clustered. Zatem silnik bazy danych mógł skorzystać z porządku określonego na `InvoiceDate` i zawęzić zakres stron do odczytu. **nonclustered** `sql CREATE NONCLUSTERED INDEX lab2_nonclustered ON dbo.lab2_sales ( CustomerID )`



W tym przypadku indeks znacząco spowalnia przeszukiwanie. Pozwala na szybsze odnalezienie poszczególnych id klientów, gdzie dla każdego id następnie trzeba znaleźć odpowiadający wiersz tabeli, aby zwrócić pozostałe pola.

nonclustered, include `sql CREATE NONCLUSTERED INDEX lab2_multi_attr ON dbo.lab2_sales ( CustomerID, InvoiceDate );`

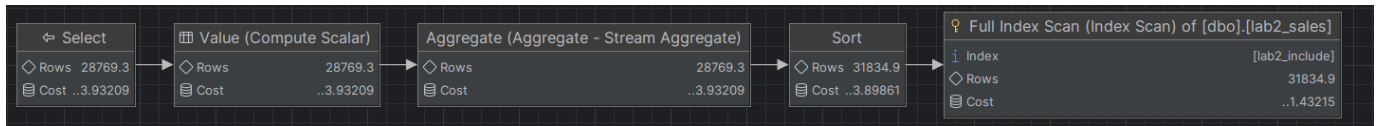


Tutaj bardzo podobny przypadek do powyższego, gdzie ustalone atrybuty nie są wystarczające, aby uzyskać wszystkie interesujące nas kolumny, więc należy dokonać dopasowań wierszy tabeli dla poszczególnych par `(customerId, invoiceDate)`.

nonclustered, include

```
CREATE NONCLUSTERED INDEX lab2_include
ON dbo.lab2_sales ( CustomerID, InvoiceDate )
INCLUDE ( StockItemID, Quantity, UnitPrice, LineProfit, ExtendedPrice );
```

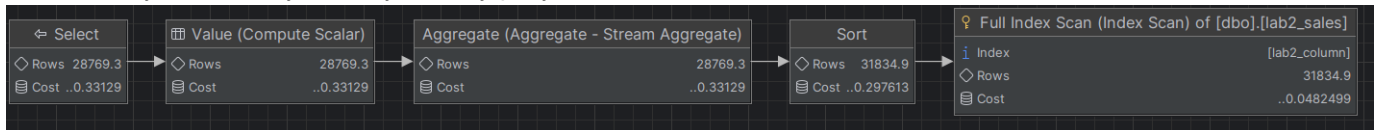
"Rozwinięcie" wcześniejszego przykładu. Teraz dla każdej pary tworzącej klucz indeksu, przyporządkowane są dodatkowe wartości kolumn (które nie są wykorzystywane do przeszukiwania w indeksie)



Zapobiega to dodatkowym lookupom, obecnym w dwóch wcześniejszych przykładach **filtered**

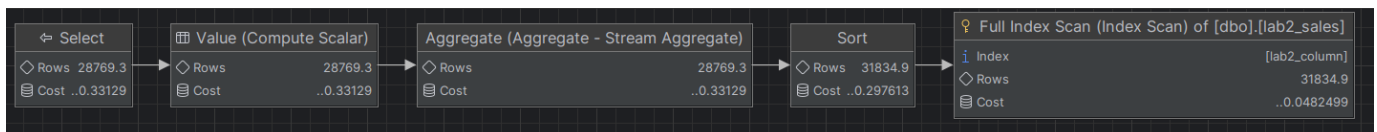
```
CREATE NONCLUSTERED INDEX lab2_filtered
ON dbo.lab2_sales ( CustomerID, InvoiceDate )
INCLUDE ( StockItemID, Quantity, UnitPrice, LineProfit, ExtendedPrice )
WHERE ExtendedPrice >= 1000;
```

Jest to indeks, który indeksuje dane zgodnie z pewnym filtrem. Ma zastosowanie, gdy mamy pewien dobrze zdefiniowany zakres danych, który chcemy pozyskać.



Plan wykonania nie zmienił się drastycznie względem wcześniejszego przykładu - niemniej można było natychmiast wykluczyć pewien zakres danych z przeszukiwania. **columnstore**

```
CREATE NONCLUSTERED COLUMNSTORE INDEX lab2_column
ON dbo.lab2_sales
(
    CustomerID,
    InvoiceDate,
    StockItemID,
    PackageTypeID,
    Quantity,
    UnitPrice,
    LineProfit,
    ExtendedPrice
);
```



Plan wykonania znów nie różni się znacząco od najlepszych indeksów. Niemniej otrzymujemy znacznie poprawiony koszt w przypadku indeksu kolumnowego.

indeks	koszt	czas [ms]	odczyty logiczne
domyślny	4.30	32.33	1818
clustered	2.84	22.33	653
nonclustered	14.61	48	220918
multi-attribute	3.93	48.66	220899
include	3.93	39	1604
filtered	2.42	17.66	403
columnstore	0.33	13,66	148

Wyszukiwanie konkretnych wartości

Wiele przykładów w tym raporcie może sugerować o dominacji indeksów kolumnowych nad pozostałymi rodzajami indeksów. Rzeczywiście radzą sobie bardzo dobrze w przypadku operacji na dużych zakresach danych. Postanowiliśmy jeszcze przetestować przypadek, gdzie wyszukujemy konkretnych wartości (co może mieć miejsce np. przy operacjach CRUD); zmodyfikowaliśmy poprzednie zapytanie do postaci:

```
SELECT CustomerID,
       StockItemID,
       sum(Quantity)           AS TotalQuantity,
       avg(UnitPrice)          AS AvgUnitPrice,
       sum(Quantity * UnitPrice) AS AvgProductTotal,
       sum(LineProfit)         AS TotalProfit,
       sum(ExtendedPrice)      AS TotalRevenue
FROM   dbo.lab2_sales WITH ( INDEX ( lab2_multi_attr ))
WHERE  CustomerId = 500
       AND InvoiceDate = '2015-01-01'
GROUP BY CustomerID, StockItemID
ORDER BY CustomerID, StockItemID;
```

Zatem filtry nie dotyczą zakresów, lecz konkretnych wartości. Dla indeksu

```
CREATE NONCLUSTERED INDEX lab2_multi_attr
ON   dbo.lab2_sales ( CustomerID, InvoiceDate );
```

koszt wynosi 0.003, a dla indeksu kolumnowego 0.72. Zatem przy wyborze indeksu należy mieć na uwadze rodzaj zapytań, które będą zazwyczaj wykonywane.

zadanie	pkt
1	6
2	2
3	5 (3 pkt. za eksperymenty + 2 dodatkowe za ciekawe/oryginalne przykłady)
razem	13

Referencje